ml

Tanvir

(data + learning algorithm) $\rightarrow$ function

1 Supervised learning

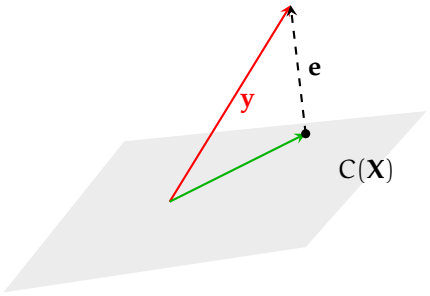
1.1 Regression

model, parameters, cost function, objective

1.1.1 Linear (in w) Regression

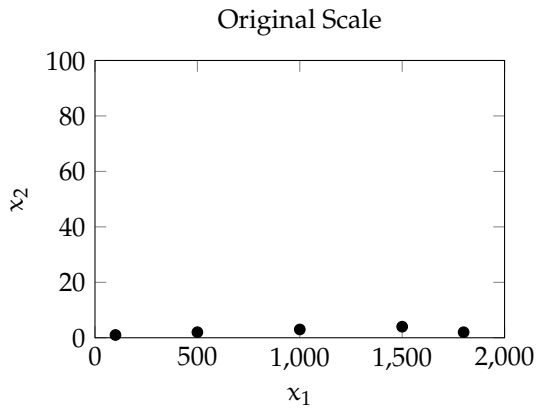
$\mathbf{X}^{m \times n}$ has m examples, each having n features. Usually $m \gg n$. $\mathbf{y}^{m \times 1}$ are the corresponding outputs.

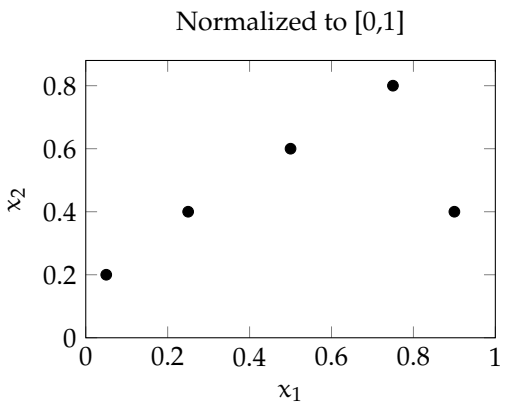
$$
\begin{aligned}
\mathbf{X}\mathbf{w} &\approx \mathbf{y} \\
\text{error}(\mathbf{w}) &= \mathbf{X}\mathbf{w} - \mathbf{y} \\
\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w}) &= \frac{1}{m}\|\text{error}(\mathbf{w})\|^2 = \frac{1}{m}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y}) \\
\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} &= \frac{2}{m}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) \\
\texttt{gradient descent:}\quad \mathbf{w} &= \mathbf{w} - \alpha\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}
\end{aligned}
$$

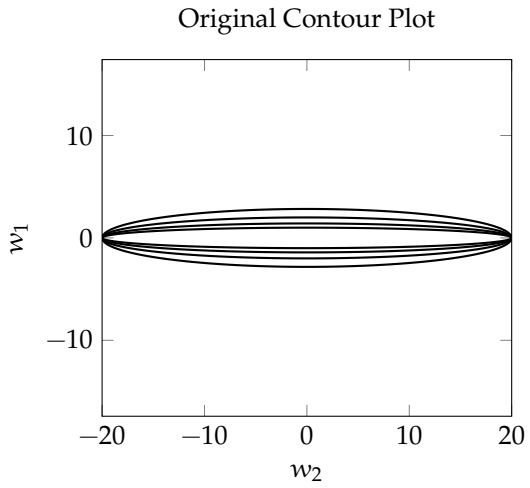


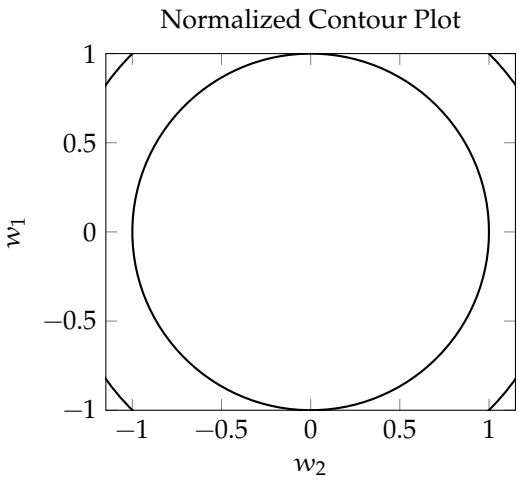
1.2 Normalization

Changes scale keeping the shape of distribution same. Gradient descent now works, with more ease, in the world of concentric circular contours.



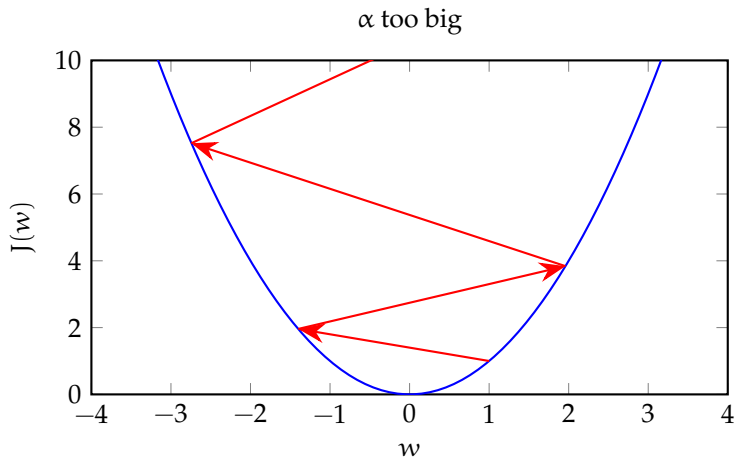






1.3 Learning rate, α

A good value is somewhere between too big (divergence) and too small (slow convergence).



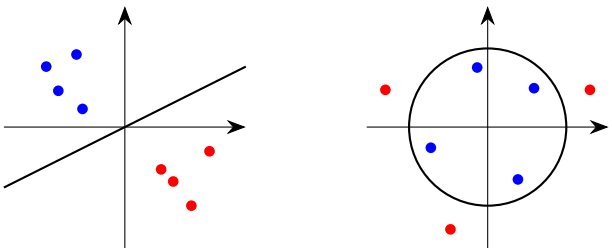
1.4 Classification

1.4.1 Logistic Regression

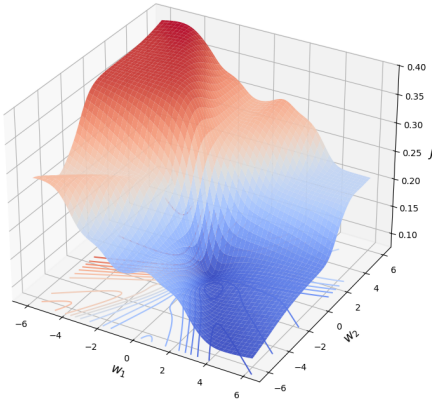
$$\mathbf{z} = \mathbf{X}\mathbf{w}, \qquad \texttt{// linear regression}$$
$$\sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}} \qquad \texttt{// maps z to probability-like (0,1)}$$

At $\mathbf{z} = \mathbf{0}$, $\sigma(\mathbf{z}) = 0.5$. So, with threshold 0.5, $\mathbf{z} = \mathbf{0}$ is the decision boundary. As linear regression doesn't have to be straight line, logistic regression's decision boundary does not have to be a straight line.

Linear boundary **Circular boundary**



The squared error loss function $L(\mathbf{w}, y_i) = \left(\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i\right)^2$, makes the cost function non-convex.

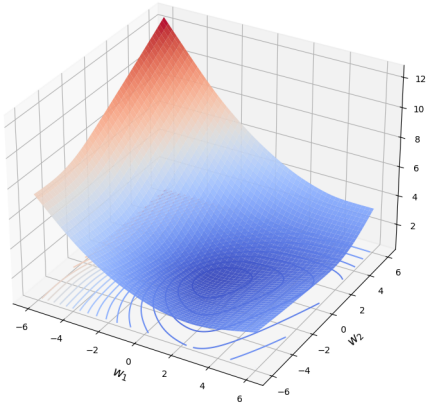


Negative log likelihood loss:

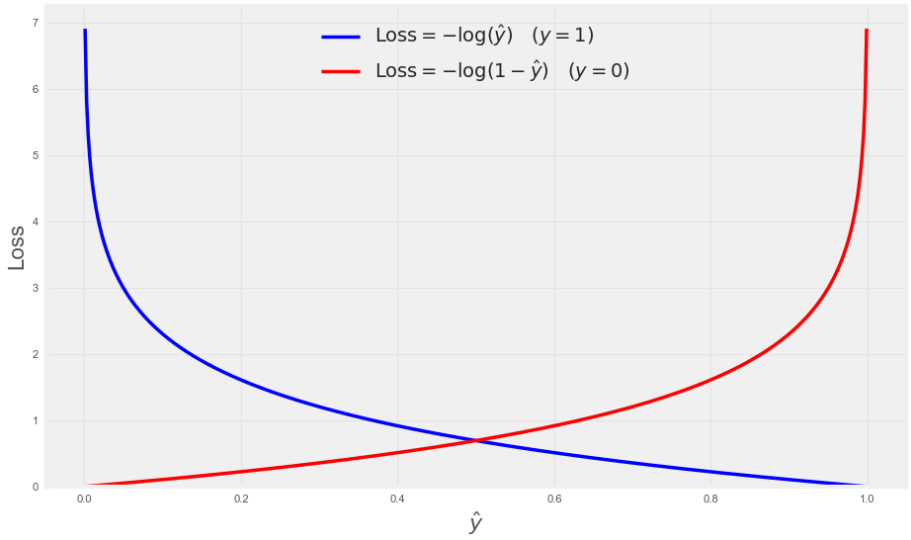
$$\hat{y}_i = \sigma\left(\mathbf{w}^T \mathbf{x}_i\right) \in (0, 1)$$

$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} \begin{cases} -\log(\hat{y}_i), & \text{if } y_i = 1, \\ -\log(1 - \hat{y}_i), & \text{if } y_i = 0 \end{cases}$$

The negative log likelihood loss function with $\sigma(z)$ makes the cost function convex.



Negative log likelihood also incentivizes appropriately: big mismatch $\implies$ big loss, small mismatch $\implies$ small loss.



$$z^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)}$$

Loss per example, $L\left(\mathbf{x}^{(i)}, \mathbf{w}\right) = -y^{(i)} \log\left(\sigma\left(z^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - \sigma\left(z^{(i)}\right)\right)$

$$\frac{\partial \sigma\left(z^{(i)}\right)}{\partial \mathbf{w}} = \frac{\partial \sigma(z^{(i)})}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \mathbf{w}} = \sigma(z^{(i)})(1 - \sigma(z^{(i)}))\mathbf{x}^{(i)}$$

$$\frac{\partial L\left(\mathbf{x}^{(i)}, \mathbf{w}\right)}{\partial \mathbf{w}} = \left(\sigma\left(z^{(i)}\right) - y^{(i)}\right) \mathbf{x}^{(i)}$$

$$\frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial L\left(\mathbf{x}^{(i)}, \mathbf{w}\right)}{\partial \mathbf{w}}$$

More concisely,

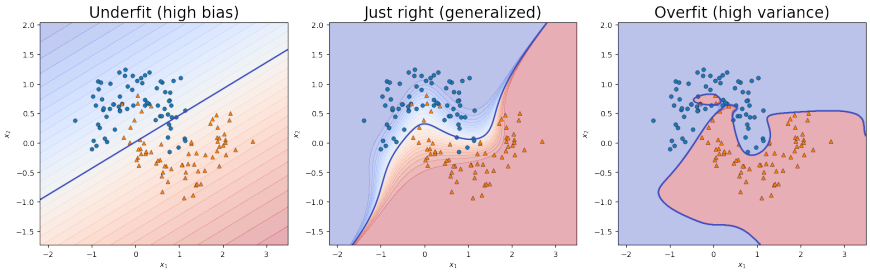
$$\mathbf{z} = \mathbf{X}\mathbf{w}$$

$$J(\mathbf{X}, \mathbf{w}) = -\frac{1}{m}\left[\mathbf{y}^T \log(\sigma(\mathbf{z})) + (\mathbf{1} - \mathbf{y})^T \log(1 - \sigma(\mathbf{z}))\right]$$

$$\frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}} = \frac{1}{m}\mathbf{X}^T(\sigma(\mathbf{z}) - \mathbf{y})$$

Gradient descent: $\mathbf{w} = \mathbf{w} - \alpha \frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}}$

2 Generalization



2.1 Address overfit

1. Collect more training examples keeping the model as it is.
2. Select a smaller set of informative features to simplify the model. Regularizarion disables features in a gradual manner.

2.1.1 Regularization

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w}) = \frac{1}{m}\|\text{error}(\mathbf{w})\|^2$$

$$\texttt{gradient descent:}\quad \mathbf{w} = \mathbf{w} - \alpha\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$$

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J_{\text{reg}}(\mathbf{w}) = \frac{1}{m}\left(\|\text{error}(\mathbf{w})\|^2 + \lambda\|\mathbf{w}\|^2\right)$$

$$\texttt{gradient descent (reg):}\quad \mathbf{w} = \underbrace{\left(1 - \alpha\cdot\frac{\lambda}{m}\right)}_{\text{shrinkage}}\cdot\mathbf{w} - \alpha\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \qquad \texttt{//}\ (\alpha\cdot{}^{\lambda}/_{m}) \sim 0$$

Here, $\lambda\|\mathbf{w}\|^2$ encourages smaller parameter values for less important features. With bigger λ's, the parameters approach zero.

We can think regularization in terms of a constrained optimization problem:

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w})$$

$$\texttt{with constraint:}\quad \|\mathbf{w}\|^2 \leqslant r^2$$

In other words, minimize $J(\mathbf{w})$ but do not let $\mathbf{w}$ go beyond a ball of radius r. The training process decreases r gradually.

3 References

- Machine learning specialization – Coursera